APP для MK61s: формат, установка и manifest

Версия документа: 20.09.2026. Исполняемый формат ABI 6.

`.APP` — единый тип нативного контейнера Cortex-M4 для STM32F401/F411. Публичный `kind` различает обычный `APPLICATION` и канонические роли FOCAL, TinyBASIC, WBMP, Markdown, CHIP-8, SETUP и USBDISK. Формат, startup, loader, API, релокации, MPU и кэш от Kind не зависят.

Для первой самостоятельной программы используйте короткий Hello World, затем полное руководство C/C++. Точный wire-формат описан в `sdk/portable/README.md`.

Размещение и совместимость

FOCAL, TinyBASIC, WBMP, Markdown и CHIP-8 независимо выбираются как встроенные, APP или выключённые. В режиме APP они имеют канонические пути `/System/FOCAL.APP`, `BASIC.APP`, `WBMP.APP`, `MARKDOWN.APP`, `CHIP8.APP`. На F411 они встроены по умолчанию, на F401 по умолчанию вынесены в APP. На F401 в `/System` также находятся `SETUP.APP` и `USBDISK.APP`. На F411 SETUP и USB-диск встроены по умолчанию. `MK61_SETUP_AS_APP=1` выносит SETUP, а `MK61_EXTERNALIZE_USBDISK=1` — USB-диск в соответствующий файл `/System`. APPLICATION можно хранить под любым допустимым M8-именем в любом каталоге C6; `/Apps` — только удобное соглашение.

Resident исполняет только ABI 6. Контейнеры ABI 2/3/4/5 нужно пересобрать; режима с точной привязкой к resident ELF/BIN или фиксированным адресом больше нет. После перехода на ABI 6 устанавливайте resident и весь `/System` из одного комплекта. Дальнейшее обновление совместимого resident не требует пересборки APP.

Размер `image + data + bss` одного APP ограничен 20 480 байтами. Это лимит контейнера, а не резерв SRAM. Linker resident экспортирует свободный диапазон после `_end` и перед stack guard, не создавая в нём массива APP. Loader выделяет сверху ровно `align32(memory_size)`; куча и staging используют низ. Неисполняемый кэш можно вытеснить, после чего весь блок возвращается пулу.

`load_address=0x20000000` в заголовке — виртуальная база линковки. Сборщик извлекает из временного ELF `R_ARM_ABS32`, записывает компактную delta-таблицу, а loader прибавляет дельту фактического адреса. Полный ELF на устройство не копируется. Payload хранится как ZX0 либо как меньший BCJ + ZX0.

Установка и запуск

Через USB-диск:

1. Откройте на приборе Меню → USB-диск.
2. Скопируйте APP в `/Apps` или другой каталог.
3. Дождитесь окончания записи и безопасно извлеките диск.
4. Нажмите `ESC`, откройте Проводник и выберите APP клавишей `0K`.

Через терминал:

```
open /Apps/HELLO.APP
run /Apps/HELLO.APP
```

При записи через USB обработчик FAT проверяет заголовок, размеры контейнера и сохранённый CRC до публикации нового дерева C6. Полную распаковку ZX0/BCJ, релокации и CRC образа выполняет обычный загрузчик при запуске: во время USB-сеанса запуск APP всё равно недопустим; кроме того, во внешнем режиме сам `USBDISK.APP` занимает исполняемую арену. Повреждённый файл не заменяет рабочий.

Manifest-сборка

`tools/firmware/stm32/build_f401_bundle.sh` умеет добавить собственные APP в полный комплект. Manifest задаёт имя, исходники и необязательный magic обработчика файла:

```
format 1
name MYAPP
source main.c
source logic.cpp
file include/logic.hpp
magic C2
```

Директивы:

Директива	Назначение
`format 1`	Версия синтаксиса manifest.
`name NAME`	Basename результата `Apps/NAME.APP`, до 31 ASCII-байта.
`source path`	`.c`, `.cpp`, `.S` или `.rs`, входящий в APP.
`file path`	Дополнительный заголовок/ресурс исходников.
`magic XX`	Необязательная двухбайтовая сигнатура типа C6 для FILE_OPEN.

Пути задаются относительно manifest, не могут быть абсолютными или выходить через `..`. Имена сравниваются без учёта регистра; FAT/DOS-имена `CON`, `PRN`, `AUX`, `NUL`, `COM1`–`COM9`, `LPT1`–`LPT9` запрещены.

Проверка manifest без сборки firmware:

```
./tools/firmware/stm32/build_f401_bundle.sh --check-app-manifests \
  --app-manifest path/to/MYAPP/app.mk61
```

Сборка комплекта:

```
./tools/firmware/stm32/build_f401_bundle.sh --profile mini-v3-a00 \
  --app-manifest path/to/MYAPP/app.mk61
```

В результате обычный APP всё равно собирается через `tools/apps/portable/build_portable_app.py` и получает ABI 6. Manifest не включает другой ABI и не открывает приватные символы resident.

Общий API

Простой APP реализует `main()` и при необходимости `mk61_app_open_file(id)`. Общий `sdk/portable/start.c` предоставляет точку входа. Продвинутый APP может переопределить `mk61_app_initialize()` и `mk61_app_command()` — те же hooks используют штатные System APP.

Базовая таблица `mk61_app_api` предоставляет экран, клавиатуру, время, звук, LED, чтение C6 и графику. `query_service` открывает всем Kind одну публичную таблицу v2: каталог и запись C6, workspace/scratch, редактор, диалоги, настройки, RTC, математику, регистры и runtime helpers. Закрытого System API нет; `loadable_system_api.h` содержит лишь алиасы для старых исходников.

Повторный запуск одного inode/заголовка использует общий opportunistic cache и может сохранить `.data`/BSS. После вытеснения startup выполняется заново. Состояние, которое обязано пережить вытеснение или перезагрузку, храните через публичный workspace либо в C6.